Aprendizaje Federado

Extensiones de Machine Learning

Ginés Carrillo Ibáñez
Yago Ibarrola Lapeña
Aarón Palomar Peña

Universidad de Murcia
Facultad de Informática

Curso 2025/2026

Contenidos

1 Sesión 1 **2**
1.1 FL centralizado 2
1.2 FL descentralizado 2

2 Sesión 2 **4**
2.1 Model poisoning 4
2.2 Label flipping 4
2.3 Comparación y análisis 5

3 Sesión 3 **6**
3.1 Trimmed Mean 6
3.2 Krum 6
3.3 Comparación y análisis 7

4 Sesión 4 **8**
4.1 Pesos estáticos 8
4.2 Pesos dinámicos 8
4.3 Comparación y análisis 9

5 Sesión 5 **10**
5.1 Conclusión 10

1 Sesión 1

1.1 FL centralizado

Comparamos el entrenamiento IID y non-IID entre los nodos. En el caso non-IID, se utiliza una distribución de dirichlet con $\alpha = 0.1$. Se puede ver la comparación de ambos casos en la Figura 1:

IID **non-IID**

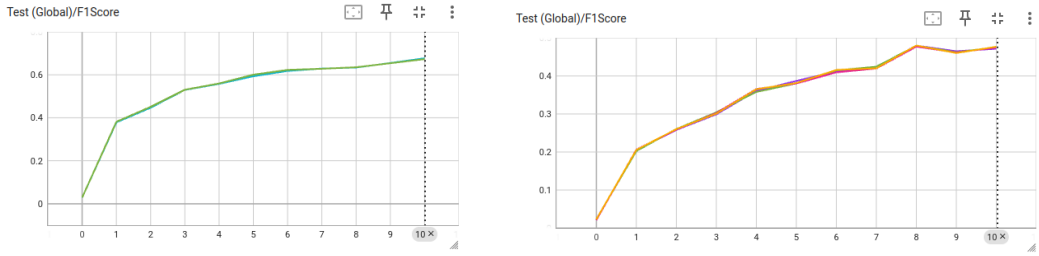

Figure 1: Comparación del entrenamiento centralizado con datos IID y non-IID.

Vemos que en el caso de datos homogéneos alcanza una mayor precisión, llegando en la ronda 10 a un F1-score de aproximadamente 0.65, mientras que en non-IID solo alcanza un F1-score de 0.47. Esto tiene sentido, ya que en el caso non-IID, cada nodo tiene una distribución de datos diferente, lo que dificulta la generalización del modelo global. Igualmente, en la ronda 3 IID ya alcanza un F1-score de 0.52, mientras que non-IID solo llega a 0.3.

1.2 FL descentralizado

En este caso, se ha realizado el mismo experimento pero con un entrenamiento descentralizado. Se muestran los resultados en la Figura 2:

IID **non-IID**

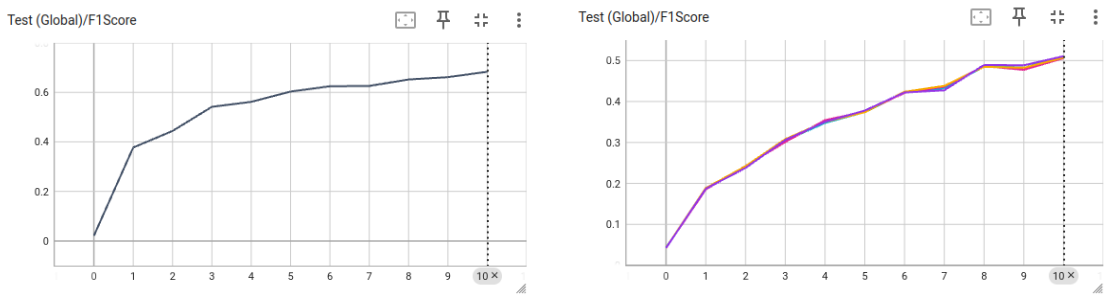

Figure 2: Comparación del entrenamiento descentralizado con datos IID y non-IID.

- Ronda 3: IID alcanza un F1-score de 0.5, mientras que non-IID solo llega a 0.3.
- Ronda 10: IID alcanza un F1-score de aproximadamente 0.67, mientras que non-IID solo alcanza un F1-score de 0.5.

En general, aunque CFL y DFL muestran resultados similares en términos de rendimiento (F1-score), CFL destaca por su simplicidad y menor sobrecarga de red, pero introduce un punto único de fallo y cuellos de botella. Por otro lado, DFL ofrece mayor resiliencia frente a fallos y ataques, a costa de una mayor complejidad en la gestión de modelos, sincronización para la agregación y comunicaciones entre nodos.

Podemos ver las matrices de confusión para ambos casos en la Figura 3:

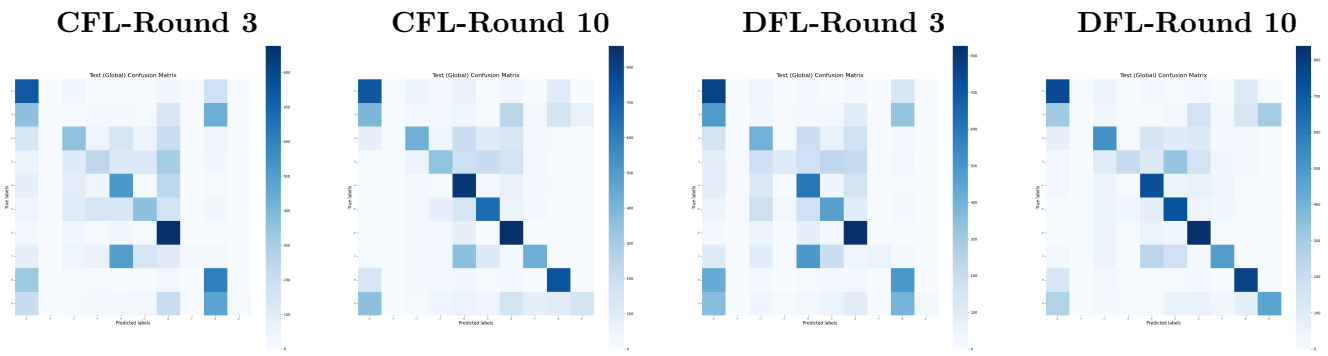

Figure 3: Matrices de confusión para el entrenamiento non-IID en CFL y DFL en las rondas 3 y 10.

Vemos que ambas rondas 3 son muy parecidas, donde todavía la diagonal no está muy marcada, tendiendo a predecir las clases 0, 4 y 8. En la ronda 10 sí que se aprecian menos errores, con una diagonal más marcada.

2 Sesión 2

2.1 Model poisoning

En este ataque, los clientes maliciosos añaden ruido gaussiano a sus modelos locales antes de enviarlos para la agregación. Este ruido degrada la calidad de las actualizaciones, afectando al proceso de convergencia. Los resultados obtenidos se muestran en la Figura 4:

20% nodos maliciosos **50% nodos maliciosos**

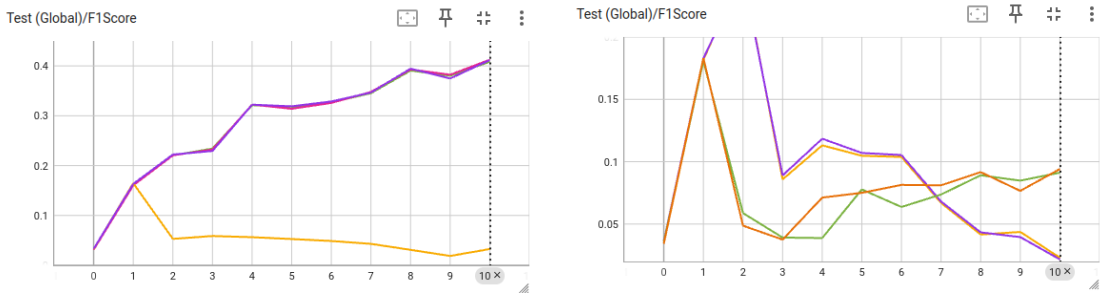

Figure 4: Impacto del ataque de *model poisoning* con diferentes porcentajes de nodos maliciosos.

Se observa que el impacto del ataque aumenta significativamente con el número de nodos maliciosos, provocando una notable inestabilidad:

- Con un 20% de nodos maliciosos, los tres nodos honestos logran converger, aunque alcanzan un F1-score de 0.4, lo que representa una caída del 10% respecto al escenario sin ataques. Por su parte, el nodo malicioso, tras recibir el modelo, introduce ruido antes de evaluarlo, obteniendo un F1-score cercano a 0 (lo que indica que su modelo ha sido corrompido). Dado que los nodos honestos agregan actualizaciones tanto de otros nodos honestos como del malicioso, el impacto del ruido introducido por este último se ve atenuado, aunque sigue provocando una degradación en el rendimiento.

- Con un 50% de nodos maliciosos, el comportamiento se degrada significativamente y hay una caída muy pronunciada. Los dos nodos benignos acaban con un F1-score cercano a 0, ya que al haber la mitad de nodos maliciosos, ha sido imposible para los modelos aprender nada útil y se han visto arrastrados por el ruido introducido.

2.2 Label flipping

En este caso, los nodos maliciosos alteran las etiquetas de un 80% de sus muestras de entrenamiento de manera aleatoria. Este ataque introduce información incorrecta directamente en el proceso de aprendizaje. Los resultados se muestran en la Figura 5:

20% nodos maliciosos **50% nodos maliciosos**

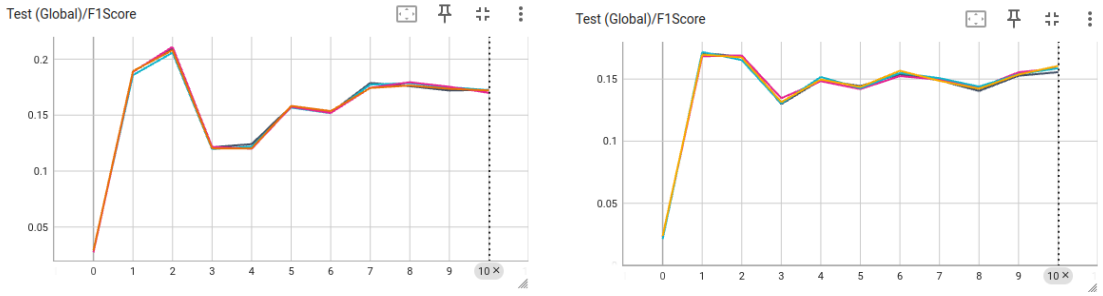

Figure 5: Impacto del ataque de *label flipping* con diferentes porcentajes de nodos maliciosos.

En este escenario, el impacto reduce el rendimiento pero mantiene un entrenamiento más estable:

- Con un 20% de nodos maliciosos, el modelo presenta una convergencia similar para todos los clientes, pero el rendimiento se desploma, alcanzando un F1-score final en torno a 0.17 (casi aleatorio pues

trabajamos con 10 clases).

- Con un 50% de nodos maliciosos,el modelo logra converger de forma similar, estancándose de manera uniforme en un F1-score de 0.16.

2.3 Comparación y análisis

Comparando ambos ataques y configuraciones con los datos obtenidos, se extraen las siguientes conclusiones:

- El ataque de *model poisoning* resulta más perjudicial que el de *label flipping*. Con un 20% de nodos maliciosos, el sistema todavía logra converger parcialmente (F1-score 0.4), pero con un 50% el modelo colapsa completamente (F1-score cercano a 0). En contraste, *label flipping* degrada el rendimiento de forma consistente pero mantiene una convergencia estable, con valores en torno a 0.17 y 0.16 para 20% y 50%, respectivamente.

- A medida que aumenta el porcentaje de clientes maliciosos, el impacto es más visible en *model poisoning* que en *label flipping*. Mientras que en el primer caso se produce una caída drástica del rendimiento (de 0.4 a 0), en el segundo la degradación es leve (de 0.17 a 0.16), lo que indica que el ruido gaussiano en los modelos afecta más gravemente al proceso de agregación que la introducción de etiquetas incorrectas.

- En la sesión anterior, el entorno DFL ideal (IID) alcanzaba un F1-score de 0.67, mientras que la introducción de heterogeneidad (DFL non-IID base) lo reducía a 0.50. Por tanto, la presencia de ataques evidencia que los entornos non-IID son especialmente vulnerables a ataques bizantinos, ya que la agregación tiene mayores dificultades para distinguir entre datos heterogéneos de perturbaciones maliciosas.

- La exposición prolongada a ataques de *model poisoning* tiene efectos variables según el porcentaje de nodos maliciosos. En el caso de 20%, apenas afecta a la velocidad de converrgencia; mientras que en el caso del 50% de nodos maliciosos, tras varias rondas de ataque, el F1-score cae drásticamente.

3 Sesión 3

3.1 Trimmed Mean

El algoritmo *Trimmed Mean* consiste en eliminar un porcentaje de los valores más extremos (potencialmente maliciosos) antes de realizar la agregación de los modelos, reduciendo así la influencia de outliers. En *Nebula*, hay que configurar el parámetro `beta` para indicar qué cantidad de valores extremos se desea eliminar. Por defecto, este valor es 0, lo que implicaba que se estaba utilizando una media simple sin recorte. Tras cambiar este valor a 1, los resultados muestran una mejora significativa, como se muestra en la Figura 6.

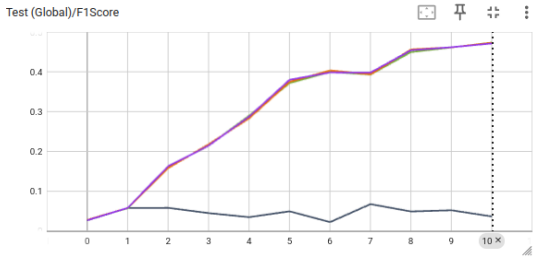

Figure 6: Resultados del entrenamiento con agregación *Trimmed Mean*.

Se observa que este método consigue anular prácticamente en su totalidad el efecto del nodo malicioso:

- El modelo converge de manera estable, evitando las oscilaciones bruscas observadas en *model poisoning* sin defensa.
- El F1-score final se sitúa en torno a 0.47, lo que supone una mejora significativa frente al caso atacado sin mecanismos de defensa (0.4).
- Sin embargo, el rendimiento sigue siendo ligeramente inferior al escenario sin ataques, indicando que el filtrado de valores extremos no elimina completamente el impacto adversario.

3.2 Krum

El algoritmo *Krum* selecciona la actualización más representativa entre los nodos, basándose en la distancia entre modelos, con el objetivo de descartar aquellos que se alejan significativamente (potencialmente maliciosos). Los resultados se presentan en la Figura 7:

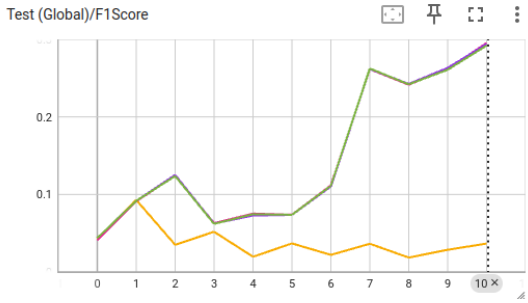

Figure 7: Resultados del entrenamiento con agregación *Krum*.

En este caso, el comportamiento es diferente:

- El entrenamiento empeora respecto al escenario sin defensa y tampoco alcanza la estabilidad observada con *Trimmed Mean*. Parece que a los nodos benignos les cuesta distinguir al nodo malicioso, lo que provoca un comportamiento aleatorio hasta la ronda 6, donde ya empieza a converger.
- El F1-score final se sitúa aproximadamente en 0.29, lejos de los valores obtenidos con *Trimmed Mean*.

- Esto sugiere que, aunque Krum es capaz de descartar actualizaciones maliciosas, puede verse afectado por la heterogeneidad non-IID, seleccionando modelos que no representan adecuadamente la distribución global.

Vemos la matriz de confusión tanto del nodo malicioso como de uno de los nodos benignos en la última ronda, en la Figura 8:

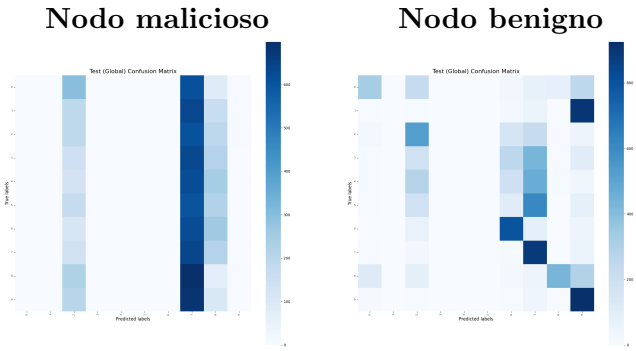

Figure 8: Matrices de confusión del nodo malicioso y uno de los nodos benignos en la última ronda con agregación Krum.

Vemos que el modelo del nodo malicoso predice únicamente las clases 2, 7 y 8. Esto hace que el nodo benigno se vea arrastrado, haciendo que tanto la clase 2 como la clase 7 se vean sobrepredichas y sin embargo hay clases como la 3, 4 y 5 que no tienen ningún verdadero positivo, porque se confunden tanto con la clase 2 como con la clase 7. Las matrices de confusión de Trimmed Mean son muy similares.

3.3 Comparación y análisis

A partir de los resultados obtenidos, se pueden extraer las siguientes conclusiones:

- *Trimmed Mean* demuestra una mayor robustez frente a ataques de *model poisoning*, ya que consigue estabilizar la convergencia y alcanzar un F1-score superior en promedio al descartar las actualizaciones extremas potencialmente maliciosas. Sin embargo, *Krum* puede verse penalizado en escenarios con datos non-IID, ya que la variabilidad entre nodos puede confundirse con comportamiento malicioso, afectando negativamente a la selección de modelos.

- Ninguno de los métodos alcanza los niveles del escenario sin ataques, lo que confirma que, aunque las técnicas robustas ayudan, no eliminan completamente el efecto adversarial.

4 Sesión 4

4.1 Pesos estáticos

Las reputaciones calculadas por cada nodo se muestran en la Figura 9:

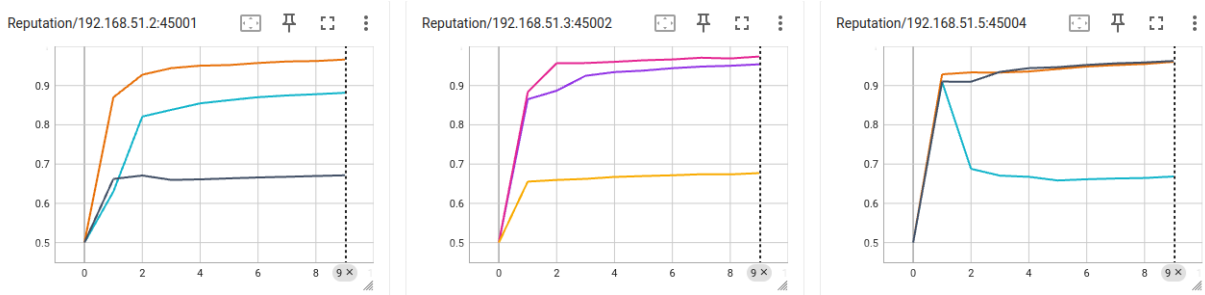

Figure 9: Reputación con pesos estáticos a lo largo del entrenamiento.

Vemos que los tres nodos honestos otorgan una reputación inferior al nodo malicioso. Desde el punto de vista de cada nodo honesto, los demás nodos honestos tienen una reputación muy cercana a 1, mientras que la reputación del nodo malicioso es algo inferior a 0.7. Esta diferencia coincide con la poderación de la métrica de número de mensajes (0.3), en la que el nodo malicioso obtiene una puntuación terrible (envía alrededor de 15 mensajes por ronda). Dado que el nodo malicioso obtiene una buena puntuación en las otras métricas, su reputación total se mantiene por encima de 0.5 y el nodo malicioso se sigue considerando como honesto.

En cuanto a los resultados del entrenamiento con esta configuración, se muestran en la Figura 10:

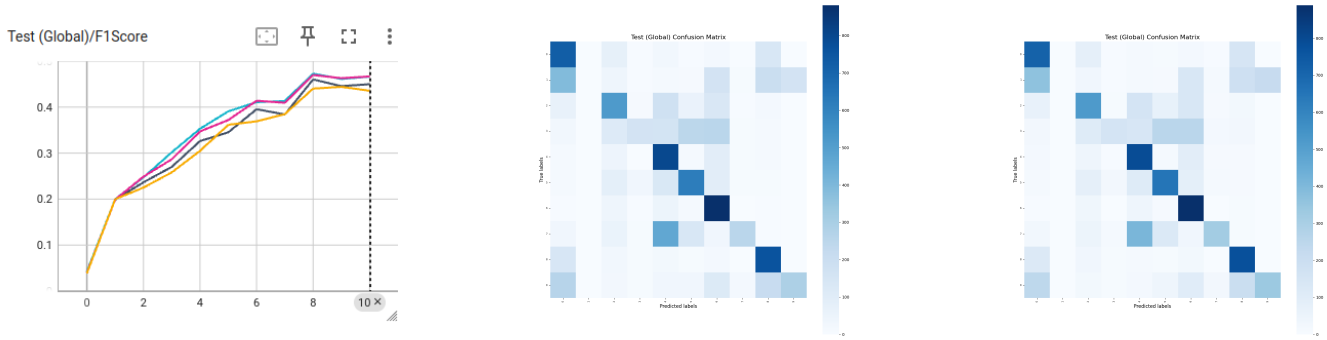

Figure 10: Resultados del entrenamiento con pesos estáticos: F1-score y matrices de confusión. Matriz de confusión del nodo malicioso a la derecha.

La gráfica del F1-score apenas muestra señales del ataque. Los modelos alcanzan un F1-score cercano a 0.46, mientras que el baseline (sin ataque) alcanza alrededor de 0.49. Esto es debido a que, a pesar del ataque de flooding, el contenido de los mensajes del nodo malicioso es correcto. El ataque resulta en un aumento del tráfico, lo que si hace que el tiempo de entrenamiento esté cerca de duplicarse (4 minutos baseline vs 8 minutos con flooding y pesos estáticos). Parte de este overhead puede ser debido también a la gestión de la reputación. En cuanto a las matrices de confusión, las cuatro resultantes son prácticamente idénticas (y muy similares a las comentadas en la Sesión 1), lo que indica que los modelos han convergido. A la izquierda se muestra la matriz de confusión de un nodo honesto y a la derecha la del nodo malicioso.

4.2 Pesos dinámicos

Los resultados se presentan en la Figura 11:

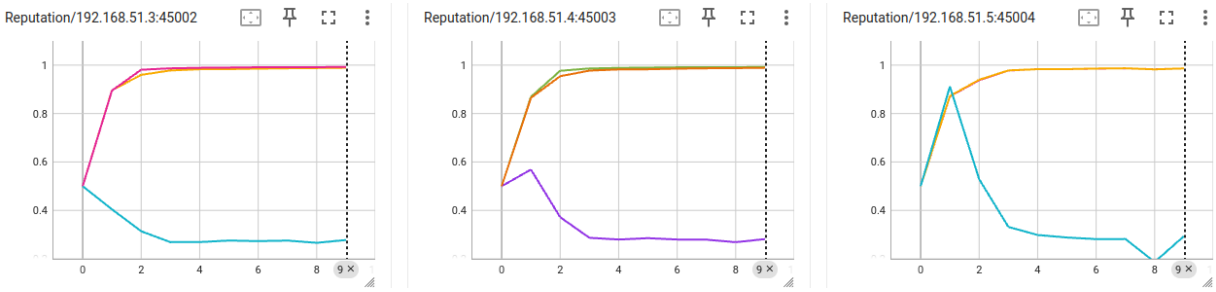

Figure 11: Reputación con pesos dinámicos a lo largo del entrenamiento.

En este caso, se observa que los nodos honestos asignan una reputación mucho más baja al nodo malicioso. Esta vez la reputación del nodo malicioso cae hasta aproximadamente 0.3. Este valor, al quedar por debajo de 0.5, causa la exclusión de las actualizaciones del nodo malicioso en el proceso de agregación. Sin embargo, parece que sí se le siguen enviando las actualizaciones del modelo global, lo que permite que el nodo malicioso siga aprendiendo y mejorando su rendimiento. Los resultados del entrenamiento con esta configuración se muestran en la Figura 12:

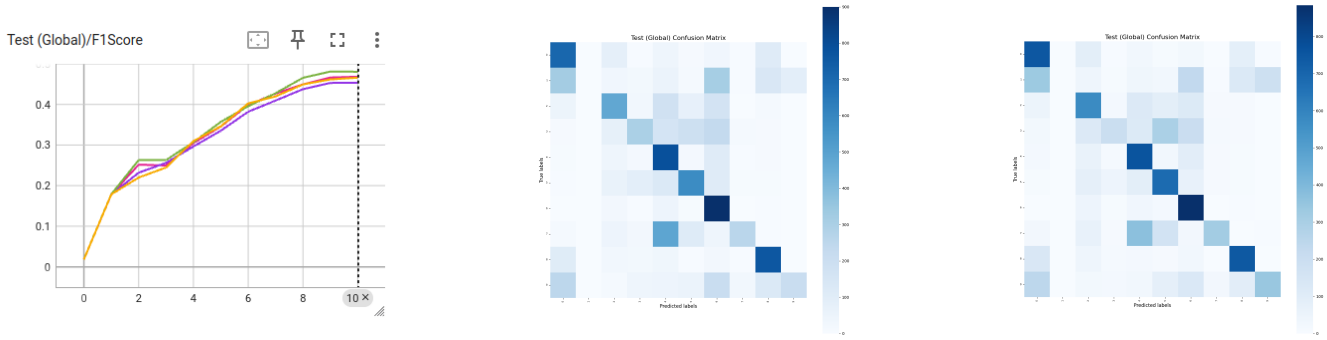

Figure 12: Resultados del entrenamiento con pesos dinámicos: F1-score y matrices de confusión. Matriz de confusión del nodo malicioso a la derecha.

La gráfica de F1-score es también similar a la anterior. Resulta curioso que el nodo malicioso obtenga un rendimiento incluso mejor que el resto, lo que se debe a que es el único que sigue utilizando sus datos para entrenar (al tener baja reputación, los otros nodos descartan sus actualizaciones). Esto, irónicamente, aporta cierta ventaja al nodo malicioso. El tiempo de entrenamiento es muy similar al de la configuración de pesos estáticos. Esto puede ser debido a que el nodo malicioso sigue enviando y recibiendo mensajes, aunque estos sean ignorados posteriormente. En cuanto a las matrices de confusión, la de la izquierda es de un nodo honesto, mientras que la de la derecha corresponde al nodo malicioso. De nuevo, los modelos convergen y las cuatro matrices de confusión son prácticamente idénticas.

4.3 Comparación y análisis

A partir de los resultados obtenidos, se pueden extraer las siguientes conclusiones:

- La reputación con *pesos dinámicos* demuestra una mayor capacidad de detección de ataques de *flooding*, y cabe esperar que también sea superior a *pesos estáticos* frente a otros tipos de ataques.
- Solo en el escenario con pesos dinámicos, el nodo malicioso es detectado y excluido completamente. Esta exclusión ocurre ya en la segunda ronda.
- La reacción de la federación ante el ataque parece ser ineficaz, incluso al detectarlo completamente. De hecho, la detección y exclusión de este causan que el modelo del atacante obtenga mejores resultados que los demás, cosa que no ocurriría si el ataque no fuera detectado.

5 Sesión 5

Compararemos la versión baseline de la sesión 1 con nuevas ejecuciones optimizadas con las técnicas de *pruning* y *quantization*.

Mostramos la evolución del F1-score para los tres casos, viendo en el eje X el tiempo total de entrenamiento:

Modelo baseline **Modelo con pruning** **Modelo con cuantización**

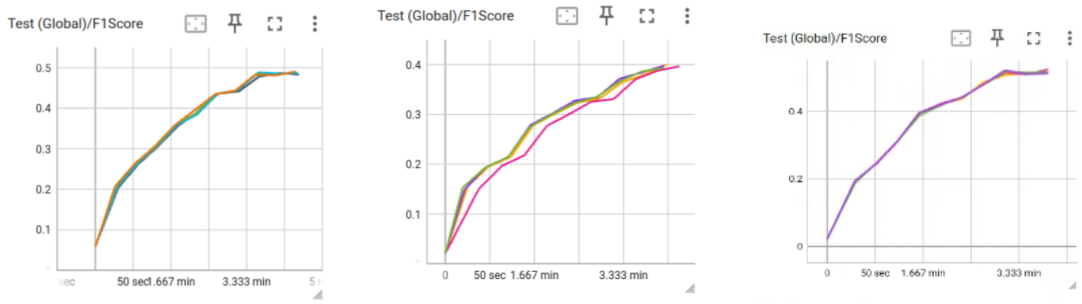

Figure 13: Impacto de técnicas de optimización en F1-score y tiempo de ejecución.

Comparando el baseline con *pruning*, vemos que claramente se ha degradado el rendimiento del modelo, pasando de un 0.48 de F1-score a estar por debajo de 0.4. Esto se debe a que el modelo es más sencillo al tener menos información en sus pesos. Sin embargo, no se ha reducido el tiempo total de entrenamiento, pues ambos tardan alrededor de 4 minutos y medio en completar las 10 rondas. El hecho de que no se mejore el tiempo creemos que se debe a que no se está utilizando ningún algoritmo de compresión específico para tensores *sparse* que permita aprovechar el *pruning*, por lo que al final se está mandando la misma cantidad de datos por la red.

Si comparamos el baseline con el modelo con cuantización vemos que sí se ha reducido el tiempo total de entrenamiento, pasando de 4 minutos y medio a 4 minutos. En cuanto al F1-score, no se ha degradado como sí ocurría en *pruning*, manteniéndose por encima de 0.5 en todos los participantes.

Cabe destacar que en estos experimentos hemos forzado el uso de GPU cambiando una línea de código en *nebula*, pues aunque nuestro ordenador con Linux nativo disponía de GPU, en los logs de entrenamiento especificaba que se estaba usando CPU, por lo que la cuantización BF16 solo se estaba simulando y no obteníamos mejoras en tiempo. Esto se debía a que en la función `get_available_gpu` de `controller.py` solo se detectaba una GPU si su uso memoria tenía un uso por debajo al 5%, lo que llevaba a descartar la GPU. Al subirlo al 50% hemos visto en los logs que sí está haciendo uso de esta. Este es el fragmento:

```
# Obtain available GPUs
if memory_used_percent < 50:
    available_gpus.append(i)
```

5.1 Conclusión

Podemos concluir que la optimización que mejores resultados ha obtenido en términos de precisión y eficiencia (pues ha tardado menos tiempo en entrenar y ha obtenido mejor F1-score) ha sido cuantización, ya que al disponer de GPU sí que aprovecha el uso de la técnica BF16 y conseguimos obtener una mejora real. Sin embargo, con *pruning* no hemos conseguido una mejora real, al no disponer de un algoritmo de compresión en el envío de los pesos que se obtienen tras realizar el *pruning*.